

FlavorNet: A Multi-Database Recipe Recommendation System

Sejma Sijaric and Lorena Raichle

sejma.sijaric@student.unisg.ch & lorena.raichle@student.unisg.ch

GitHub Repository: github.com/AdvDB-FlavorNet

Abstract

FlavorNet is a multi-database recipe recommendation system that integrates *PostgreSQL*, *MongoDB*, and *Qdrant* to deliver personalized, allergy-safe, and semantically relevant recipe suggestions. User preferences and authentication data are managed in a relational schema, recipe content is stored in a flexible document-oriented model, and semantic similarity is computed via vector search over dense embeddings. A *FastAPI* backend orchestrates these heterogeneous data stores into a unified retrieval pipeline that embeds user queries, applies database-level and semantic filtering, and hydrates results with full recipe content. The system demonstrates how complementary database technologies can be combined effectively to support preference-aware, semantics-driven recommendation workflows. The implementation is available at: github.com/AdvDB-FlavorNet

1 Introduction & Problem Description

Recipe content is widely distributed across numerous online platforms, each employing different filtering mechanisms, metadata standards, and search capabilities. Many existing recipe applications rely primarily on keyword matching, static tags, or shallow rule-based filters. As a result, recommendations often fail to capture semantic similarity between recipes and may violate users’ dietary restrictions, allergies, or personal preferences.

At the same time, user expectations have evolved. Users expect recommendation systems to understand meaning rather than exact keywords, maintain rich preference profiles, and consistently provide safe, personalized results. Meeting these requirements is challenging for systems built on a single database paradigm, as no single data model efficiently supports structured user data, flexible

recipe documents, and high-dimensional semantic embeddings simultaneously.

FlavorNet addresses this challenge by implementing a multi-database, preference-aware semantic recipe recommendation system. The architecture combines *PostgreSQL* for structured user and preference data, *MongoDB* for semi-structured recipe documents, and *Qdrant* as a vector database for efficient approximate nearest-neighbor search over dense embeddings. These components are coordinated through a *FastAPI* backend that unifies query processing across databases. The system supports user authentication, profile-based filtering (including dietary types, allergies, and ingredient exclusions), an inspiration-based recommendation feed, and semantic search over more than one million recipes from the *RecipeNLG* dataset¹. Recipe texts are embedded using a *SentenceTransformer* model to enable semantic similarity search beyond exact keyword overlap. Retrieved candidates are subsequently filtered based on user-specific constraints to ensure dietary safety and relevance. Beyond direct search, FlavorNet also addresses the challenge of inspiration-based recommendation. Traditional inspiration feeds often prioritize diversity or popularity while disregarding user-specific dietary constraints and preferences. FlavorNet combines topic-based clustering of recipes with strict preference-aware filtering, enabling exploratory recommendations that remain aligned with users’ dietary profiles and safety requirements.

This design demonstrates how heterogeneous databases can be integrated to build scalable, semantics-driven recommendation systems that respect complex user preferences.

¹<https://recipenlg.cs.put.poznan.pl>

2 System & Design Rationale

FlavorNet is designed as a multi-database system to support diverse data representations and query patterns that arise in a personalized, semantics-driven recommendation platform. The system design is guided by the following functional and non-functional requirements:

Functional Requirements

- User authentication and management of dietary preferences and allergies
- Allergy-safe recipe filtering and validation
- Semantic search over 1M+ recipes
- Inspiration-based recommendations respecting user-specific constraints

Non-Functional Requirements

- Low-latency query execution
- Scalability to > 1M recipes
- Adaptability to evolving schemas & tags
- High availability with acceptable eventual consistency
- Reproducible deployment in a containerized environment

These requirements directly informed the allocation of responsibilities across different database technologies. Rather than relying on a single data model, FlavorNet adopts a polyglot approach in which each database addresses a specific subset of requirements. The following subsections describe how *PostgreSQL*, *MongoDB*, and *Qdrant* jointly satisfy these requirements by supporting structured user data, flexible recipe content, and large-scale semantic similarity search, respectively.

Relational Schema (PostgreSQL) - Structured user data

PostgreSQL is used to store user accounts and preference profiles, including dietary types, allergies, and disliked ingredients. This data is highly structured, consistent across users, and central to enforcing safety constraints throughout the recommendation pipeline. A relational model is well suited for this purpose, as it provides strong consistency guarantees, schema enforcement,

and transactional integrity. User-related data in FlavorNet is relatively stable over time: user accounts and preference profiles follow a fixed structure and change infrequently compared to recipe content and filtering metadata. This makes a relational model with an explicit, enforced schema particularly suitable for managing user information. User preference data is accessed on every recommendation request and must be reliable and normalized. *PostgreSQL* supports efficient indexing on frequently accessed fields and ensures that each user has a single authoritative preference profile. As a result, *PostgreSQL* serves as the system of truth for all user-related information, forming the foundation upon which downstream filtering in other databases depends.

PostgreSQL was also chosen due to prior team experience with relational database design and indexing, which reduced implementation risk while ensuring desired non-functional requirements.

NoSQL (MongoDB): Semi-Structured Recipe Documents

Recipe data in FlavorNet exhibits high structural variability and schema volatility. Recipes differ widely in the number of ingredients, preparation steps, available metadata, associated tags, and new dietary categories or filtering attributes may be introduced over time. Such characteristics make NoSQL databases well suited for managing recipe data, as they are designed to handle semi-structured, heterogeneous, and evolving data models while prioritizing availability and horizontal scalability over strict schema enforcement.

Among NoSQL paradigms, a document-oriented database is particularly appropriate for this use case. Unlike key-value stores, which offer limited query expressiveness, or column-family stores, which are optimized for uniform, tabular access patterns, a document store allows complex, nested objects to be stored and retrieved as a single logical unit. This aligns naturally with the conceptual structure of a recipe as a self-contained entity and avoids the need for costly joins across multiple tables or collections.

Recipes are inherently heterogeneous and semi-structured, as they include nested ingredient lists, variable-length instructions, optional

nutritional information, and multiple tag fields. *MongoDB* is therefore used to store full recipe documents in a flexible, document-oriented format. Modeling this structure in a relational schema would require multiple join tables, whereas *MongoDB* allows the entire recipe to be stored and retrieved as a single document. Listing 1 illustrates the document-oriented structure used in *MongoDB*.

```
{
  "title": "Jewell Ball's Chicken",
  "slug": "jewell-ball-s-chicken",
  "allergen_tags": ["dairy"],
  "ingredient_tags": [
    "boned-chicken-breasts",
    "carton-sour-cream",
    "cream-of-mushroom-soup"
  ],
  "ingredients": [
    { "name": "chipped beef", "qty": 1 },
    { "name": "chicken breasts", "qty": 4 },
    { "name": "cream of mushroom soup", "qty": 1 },
    { "name": "sour cream", "qty": 1 }
  ],
  "steps": [
    "Place beef on bottom of dish.",
    "Place chicken on top.",
    "Mix soup and cream.",
    "Bake at 275 for 3 hours."
  ]
}
```

Listing 1: Example *MongoDB* recipe document

The *RecipeNLG* dataset is used as the primary recipe corpus due to its large scale and rich coverage of real-world, web-scraped recipes, including titles, ingredient lists, and step-by-step preparation instructions. The dataset is preprocessed prior to insertion to normalize ingredient strings and extract dietary and allergen tags. Each cleaned recipe is stored as a single JSON-like document, allowing optional and sparse fields to coexist without imposing rigid schema constraints. This design enables the recipe representation to evolve over time as new tags, dietary categories, or metadata are introduced, without requiring costly schema migrations.

MongoDB is optimized for read-heavy workloads and supports efficient retrieval of complete documents by identifier, which is critical during the hydration phase following vector search. Secondary indexes on fields such as ingredient tags, dietary tags, and allergen tags allow fast exclusion-based filtering without full collection

scans. While *MongoDB* provides weaker consistency guarantees than a relational system, this trade-off is acceptable in this context, as recipe data is largely read-only and the system prioritizes availability and low-latency access. Within the recommendation pipeline, *MongoDB* serves as the canonical source of truth for recipe content, enriching vector search results with complete and validated recipe data before they are returned to the user.

Lastly, *MongoDB* was selected to deepen practical experience with document-oriented NoSQL systems introduced during the course lab sessions.

Vector Database (Qdrant) - Semantic Vector Search

Relational and document databases are well suited for structured filtering and content storage but are not designed to support similarity search over high-dimensional embedding spaces. To enable semantic retrieval that goes beyond exact keyword matching and scales to more than one million recipes, FlavorNet integrates a dedicated vector database.

Qdrant stores dense vector embeddings generated for each recipe, capturing both the semantic meaning of the dish and aspects of its preparation. These embeddings are indexed using an approximate nearest-neighbor structure based on cosine similarity, enabling efficient top-*k* retrieval under low-latency constraints. Rather than applying a fixed similarity threshold, FlavorNet intentionally relies on top-*k* retrieval to always return a bounded set of candidate recipes. This design choice reflects the system’s goal of supporting inspiration-based recommendations, where relative semantic similarity is sufficient and overly strict thresholds could suppress exploratory results.

In addition to embeddings, *Qdrant* maintains a rich metadata payload for each recipe, including dietary tags, allergen tags, ingredient tags, and topic identifiers. These payloads enable database-level filtering during vector search, ensuring that semantic retrieval respects user-specific dietary constraints. Topic identifiers are derived using BERTopic and group recipes into semantically coherent clusters, such as cuisines or dish categories, which are used to support

inspiration-based recommendations.

Qdrant was selected over managed vector database services due to its support for self-hosted deployment and seamless integration into a Docker-based architecture. This choice allows the entire system to be deployed locally and reproducibly while still providing production-grade vector indexing and filtering capabilities. With this design, Qdrant directly addresses the system’s functional requirements for semantic search over more than one million recipes and for inspiration-based recommendations that consistently respect user-specific dietary constraints.

Backend (FastAPI) - Databases coordination

FastAPI serves as the central coordination layer between all databases and the user interface. It embeds user queries, retrieves user preferences from PostgreSQL, constructs filtered vector queries for Qdrant, hydrates results from MongoDB, and applies strict post-filtering to ensure dietary safety. FastAPI is well suited for this role due to its asynchronous execution model, clean separation of routing and service layers, and seamless integration with Python-based machine learning and database libraries. Its lightweight nature and compatibility with containerized deployment further support the system’s portability and reproducibility across development environments. This backend orchestration allows FlavorNet to unify heterogeneous databases into a single, coherent recommendation pipeline.

Overall, the system architecture is driven by non-functional requirements such as performance, scalability, adaptability, availability, and deployment constraints. By selecting different databases for different responsibilities, FlavorNet avoids forcing all data into a single model and instead leverages each technology where it performs best.

3 System Overview / Architecture

FlavorNet follows a layered, service-oriented architecture that integrates multiple specialized databases into a single recommendation pipeline. The system is composed of a frontend application, a backend orchestration layer, and three database systems, each responsible for a distinct category of data. Figure 1 illustrates this architecture and the corresponding data flow. The diagram shows

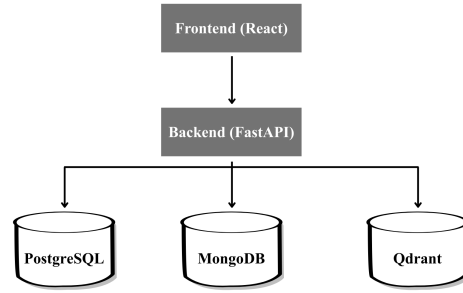


Figure 1: Architecture Diagram

the frontend communicating exclusively with the FastAPI backend, which acts as a central coordination layer. PostgreSQL supplies authoritative user preference data, Qdrant performs filtered semantic retrieval, and MongoDB provides complete recipe documents.

The frontend, implemented using *React* and *Vite*, provides the user interface for registration, authentication, profile management, inspiration browsing, and semantic search. All user interactions are translated into HTTP requests that are sent to the backend. The frontend itself does not access any database directly and relies entirely on the backend to enforce business logic and safety constraints.

While Figure 1 presents the static system architecture, Figure 2 details the dynamic recommendation pipeline executed for each user request.

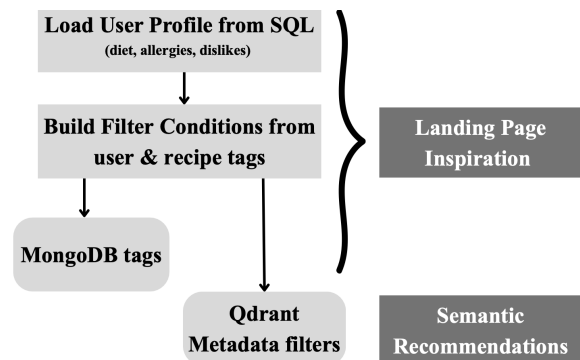


Figure 2: Recommendation Pipeline

At the core of the system is the FastAPI backend, which exposes RESTful endpoints and hosts a central *RecommendationService*. As illustrated in Figure 2, each recommendation or search request follows a fixed pipeline. The backend first retrieves the user’s preference profile from PostgreSQL,

including dietary types and allergen exclusions, which define strict constraints for all subsequent steps.

Based on the user request, the backend constructs a filtered semantic query and forwards it to the Qdrant vector database. Qdrant performs approximate nearest-neighbor search over recipe embeddings while simultaneously applying payload-based filters derived from the user’s preferences. This step yields a bounded set of candidate recipe identifiers that are both semantically relevant and compatible with the user’s dietary constraints.

The resulting identifiers are subsequently used to retrieve full recipe documents from MongoDB. This hydration phase enriches the vector search results with complete recipe content, including ingredient lists, preparation steps, and associated metadata. Finally, the backend performs validation and post-filtering before returning the personalized recipe recommendations to the frontend.

This pipeline ensures that all recommendations are generated through a consistent sequence of preference enforcement, semantic retrieval, and content hydration.

4 Implementation

Implementation of Semantic Indexing and Topic-Based Recommendations with Qdrant

Qdrant forms the core of FlavorNet’s semantic recommendation engine. Its primary role is to enable similarity-based retrieval that captures the meaning of recipes rather than relying on exact keyword matches. To achieve this, each recipe is represented using dense semantic embeddings generated offline. Embeddings are computed using a *SentenceTransformer* model (all-MiniLM-L6-v2, 384 dimensions), with two complementary representations: one embedding vector for the combined recipe title and preparation steps (v_{text}) and a second embedding for the ingredient list ($v_{ingredients}$). These embeddings are stored in Qdrant and indexed using an approximate nearest-neighbor structure based on cosine similarity.

To support inspiration-based recommenda-

tions, topic identifiers are derived using BERTopic and stored as metadata for each recipe. In the absence of explicit cuisine or category labels in the RecipeNLG dataset, topic modeling is used to automatically discover semantically coherent groups of recipes based on their textual descriptions. These topic assignments are not used as strict filtering constraints but instead enable exploratory recommendation modes, such as suggesting additional recipes from related semantic categories, while preference-based filtering continues to rely on dietary and allergen tags.



Figure 3: Example topic clusters discovered using BERTopic. Each cluster groups semantically related recipes and is used to support inspiration-based recommendations.

Topic modeling is performed offline on a large sample of recipe texts, combining recipe titles and preparation steps as input. Rather than training a supervised classifier, BERTopic leverages sentence-level embeddings together with clustering to identify recurring semantic themes in the corpus. This process yields interpretable clusters such as *baked goods*, *pasta dishes*, *salads*, or *tacos*, as illustrated in Figure 3. Each recipe is assigned a topic identifier and an associated confidence score, which are stored alongside the semantic embeddings in Qdrant and reused at query time to support inspiration-driven exploration.

System Implementation and Deployment

FlavorNet is implemented as a containerized system in which all components are deployed using Docker Compose. This setup ensures

reproducibility and allows the full application stack, including the FastAPI backend, React (Vite) frontend, PostgreSQL, MongoDB, and Qdrant, to be launched consistently in a local development environment.

The frontend is implemented separately and communicates with the backend exclusively via HTTP APIs. The backend application is implemented using FastAPI and serves as the integration layer between all databases and the frontend. Rather than embedding database-specific logic in the frontend, all recommendation and filtering logic is executed server-side. This design centralizes safety checks and ensures consistent enforcement of user preferences across all retrieval modes.

Offline preprocessing is used to prepare the recipe corpus prior to deployment. The RecipeNLG dataset is cleaned and normalized, dietary and allergen tags are extracted, and semantic representations are generated. At runtime, the backend focuses exclusively on query orchestration: retrieving user preferences, executing filtered semantic search, hydrating results with full recipe documents, and performing final validation before returning results to the user.

To work with the system locally, FlavorNet provides a simple container-based setup that requires no manual database configuration. After cloning the repository, the entire stack can be started using Docker Compose. The first command in Listing 2 launches the backend service, all databases, as well as the frontend application, making the system immediately usable for development and testing. The frontend can then be accessed through `http://localhost:5173`, and the application can be used end-to-end.

```
# Start the full application stack
docker compose up --build -d

# Frontend (UI)
http://localhost:5173

# Optional: stop the system
docker compose down
```

Listing 2: Running the FlavorNet system locally

Listing 3 shows some commands for health checks of the services that can be performed or commands

to look at the logs of the backend, since it is the central coordination of the whole system. Alternatively, if one wants to view all the logs when starting the application, simply leave out the `-d` part from the initial command in Listing 2.

```
# Backend health check
curl http://localhost:8000/

# Qdrant service health
curl http://localhost:6333/health

# Inspect backend logs
docker compose logs backend
```

Listing 3: Service health checks and logs

5 Evaluation & Results

FlavorNet is evaluated as an end-to-end multi-database recommendation system. The evaluation focuses on three core goals: (i) correct orchestration across heterogeneous data stores, (ii) strict enforcement of user-specific dietary and allergen constraints, and (iii) semantically meaningful retrieval at scale. Rather than benchmarking individual components in isolation, we evaluate the system through representative user scenarios and the corresponding database queries executed at each stage of the recommendation pipeline.

System Integration and Completeness

A central correctness criterion is that the system consistently returns complete and validated recipe objects, even though candidate selection may originate from different databases. In semantic search mode, candidates are retrieved from Qdrant based on vector similarity, while inspiration-based feeds may originate from MongoDB using tag-based filters. In all cases, the backend hydrates the final response from MongoDB using shared recipe identifiers.

This canonical hydration step ensures consistent fields (e.g., title, ingredients, steps, tags, images) and stable identifiers even when vector payloads are incomplete. To improve robustness, vector queries intentionally over-fetch candidates and prune them after hydration based on user constraints. If vector search or embedding generation fails, the system deterministically falls back to a MongoDB-only retrieval path using the same preference filters. This guarantees uninterrupted service while preserving safety guarantees.

Preference Safety (Constraint Enforcement)

Preference safety was evaluated using user profiles with strict dietary and allergen constraints. User preferences stored in PostgreSQL are translated into payload filters for Qdrant, query predicates for MongoDB, and finally revalidated after hydration. In addition to tag-based filtering, ingredient text is scanned to catch cases where allergens or non-compliant items are not explicitly tagged.

For example, for a user profile configured as *vegan* with a *peanut* allergy, vector hits missing allergen tags are discarded during post-filtering. Only recipes whose MongoDB documents still satisfy vegan and peanut-free constraints are returned. This layered enforcement ensures that unsafe recipes are never recommended, even in the presence of incomplete metadata.

Semantic Relevance (Vector Retrieval Quality)

Semantic relevance was assessed using natural-language queries where keyword overlap is limited or ambiguous. FlavorNet stores two 384-dimensional cosine embeddings per recipe and performs similarity search over the text-based representation.

For example, the query “*blueberry cake*” returns recipes such as *Blueberry Lemon Streusel Cake Recipe* and *Blueberry Cheesecake Bar*. These results demonstrate that retrieval is driven by semantic similarity rather than exact keyword matching. Importantly, semantic relevance is preserved while deterministic preference filters continue to exclude unsafe results, illustrating that vector retrieval and rule-based filtering can operate together effectively.

Representative Query Scenarios

To illustrate how user-facing problems are solved through concrete database queries, we present selected examples from the recommendation pipeline.

Hydrating Vector Search Results

- **Problem:** Convert vector search hits into complete, user-facing recipe objects.
- **Query:** After receiving candidate slugs from Qdrant, the backend retrieves full recipe documents from MongoDB.

```
db.recipes.find(
  { "slug": { "$in": [
    "blueberry-cheesecake-bar",
    "blueberry-lemon-streusel-cake-recipe"
  ] } },
  {
    title: 1,
    ingredients: 1,
    steps: 1,
    dietary_tags: 1,
    allergen_tags: 1,
    images: 1,
    nutrition: 1
  }
})
```

- **Outcome:** Fully hydrated recipes are validated against user preferences and returned to the client.

Preference-Safe Fallback Without Vector Search

- **Problem:** Provide safe recommendations when vector search is unavailable.
- **Query:** Execute a deterministic MongoDB query using the same dietary and allergen constraints.

```
db.recipes.find({
  dietary_tags: { "$all": ["vegan"] },
  allergen_tags: { "$nin": ["peanut"] },
  ingredient_tags: { "$nin": ["peanut"] }
})
.sort({ "rating.value": -1 })
.limit(12)
```

- **Outcome:** A peanut-free vegan feed is returned, preserving safety guarantees without vector search.

6 Reflection & Lessons Learned

While the overall architecture proved effective, implementing a multi-database recommendation system revealed several practical challenges that are not apparent in single-database designs. The most significant challenge was maintaining consistency across heterogeneous and independent data stores. Since recipe metadata is partially duplicated between MongoDB and Qdrant, keeping these representations aligned required disciplined use of shared unique identifiers and carefully controlled preprocessing pipelines. In particular, whenever recipe content or tags change, embeddings and

payload metadata must be regenerated to ensure that semantic search results remain accurate and consistent.

A second lesson concerned the limitations of metadata-based filtering in isolation. While Qdrant’s payload filters efficiently enforce many dietary and allergen constraints, they cannot guarantee safety when certain ingredients appear only in unstructured text fields. As a result, preference constraints are enforced across multiple stages of the pipeline: during vector retrieval, during MongoDB hydration, and through final post-filtering before response delivery. Although this multi-stage validation increases system complexity, it proved essential for maintaining strong safety guarantees in a real-world dataset with incomplete or noisy annotations.

From a design perspective, an important lesson was the value of decoupling data flow across system components. User preferences, recipe metadata, and semantic embeddings are processed and evolved independently, allowing each subsystem to scale and change without tightly coupling to the others. The hybrid search strategy, combining deterministic tag-based filtering with semantic vector retrieval, emerged as a particularly effective pattern. Tags provide fast and reliable exclusion guarantees, while vector search contributes semantic relevance and supports exploratory, inspiration-driven recommendations. Ensuring that every recipe shares a stable unique identifier across all databases further simplified cross-database consistency and result aggregation.

Performance tuning also highlighted key trade-offs inherent in vector-based retrieval. Retrieving too few candidates risks losing relevant results after strict filtering, while over-fetching increases latency. Through experimentation, we identified stable defaults that balance recall and responsiveness. Additionally, implementing a fallback mechanism reinforced the importance of graceful degradation. When Qdrant is unavailable or returns no valid results under strict constraints, the system can still produce safe recommendations using MongoDB-based retrieval, preserving availability for safety-critical users.

An important and unexpected lesson was that database design decisions alone were not

sufficient to build a meaningful and relevant recommendation system. While the choice and integration of PostgreSQL, MongoDB, and Qdrant addressed structural and scalability requirements, the usefulness of the system ultimately depended on how the underlying data was prepared and represented for each database. Working with heterogeneous data formats revealed that substantial preprocessing was required to make the data compatible with different retrieval paradigms. In particular, the RecipeNLG dataset does not provide explicit cuisine or category labels that are well suited for inspiration-based exploration. As a result, natural language processing techniques, such as topic modeling with BERTopic, became essential to extract higher-level semantic structure from unstructured recipe text. This preprocessing step was crucial to fully leverage the capabilities of the vector database and highlights that modern database systems often rely on upstream machine learning and NLP components to unlock their full potential.

If the project were restarted, several improvements would be prioritized. First, automated consistency checks between MongoDB documents and Qdrant payloads, such as periodic integrity validation or lightweight background synchronization jobs, would help detect drift early. Second, caching frequently used query embeddings would reduce repeated computation and further improve response times for common searches. Overall, the project provided valuable hands-on experience with the coordination, validation, and performance challenges that arise when integrating multiple specialized databases into a single coherent recommendation system.

7 Conclusion

FlavorNet demonstrates how heterogeneous database technologies can be combined to implement a preference-aware, semantics-driven recipe recommendation system at scale. The project integrates three specialized data stores: PostgreSQL for strongly consistent user identity and preference profiles, MongoDB as the canonical store for semi-structured recipe documents, and Qdrant for efficient approximate nearest-neighbor search over dense semantic embeddings with metadata-based filtering. A FastAPI backend orchestrates these systems into a unified retrieval

pipeline that supports both semantic search and inspiration-based exploration while enforcing dietary safety constraints.

The key system-level finding is that combining deterministic filtering with vector similarity retrieval yields recommendations that are both relevant and safe. In practice, user constraints retrieved from PostgreSQL are translated into database-level filters applied during vector search and document retrieval, while MongoDB hydration ensures that results are always returned as complete recipe objects. The evaluation illustrates that semantic queries such as “blueberry cake” retrieve conceptually related recipes beyond exact keyword overlap, and that strict dietary constraints (e.g., vegan with peanut allergy) are consistently enforced through multi-stage validation. Concrete query scenarios further highlight the system’s multi-database behavior: Qdrant produces candidate identifiers via similarity search, MongoDB hydrates these identifiers into full documents, and a deterministic MongoDB fallback path preserves availability when vector search is unavailable.

Beyond the application itself, FlavorNet provides a practical case study of polyglot persistence and cross-database orchestration. The project highlights the importance of stable shared identifiers, canonical hydration to avoid fragmented truth across stores, and layered validation to mitigate incomplete metadata in real-world datasets.

Overall, FlavorNet shows that a carefully designed multi-database architecture can support modern recommendation workloads by jointly optimizing for structured preference management, flexible content storage, and scalable semantic retrieval.

Appendix

This appendix presents a step-by-step **User Interface Walkthrough** of the FlavorNet user interface, illustrating the main user interactions from authentication to personalized recipe recommendations.

- Figure 4: Login and Landing Page
- Figure 5: Create Account
- Figure 6: Edit Taste Profile

• Figure 7: Inspiration Feed

• Figure 8: Semantic Search Results

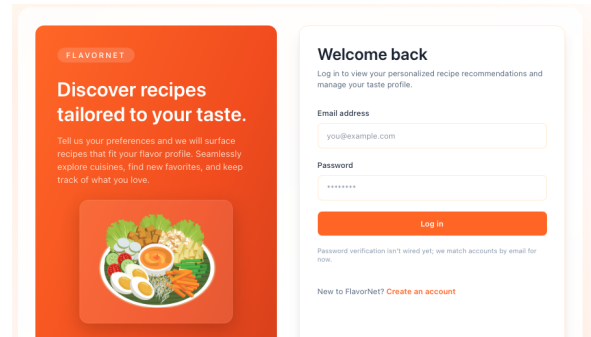


Figure 4: **Login and Landing Page.** User authentication and initial landing page. Users can sign up or log in to access personalized recommendations.

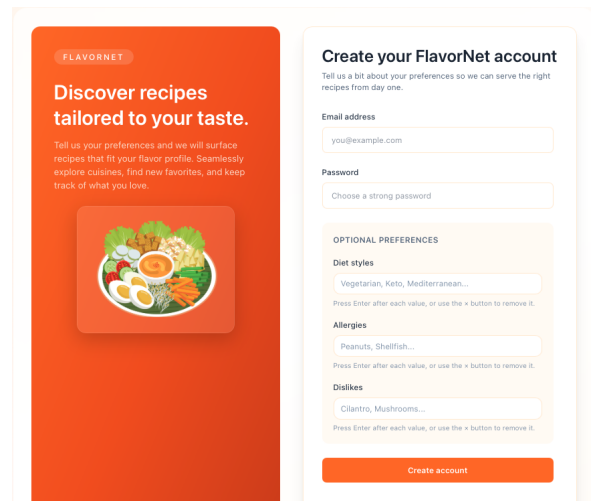


Figure 5: **Create Account.** Users can create a FlavorNet account and optionally define dietary preferences, allergies, and dislikes during signup or later in the profile section.

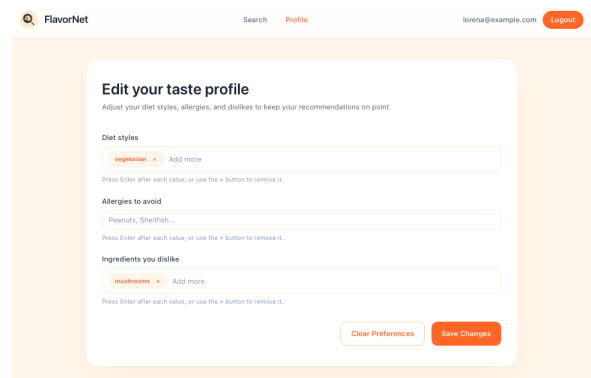


Figure 6: **Edit Taste Profile.** Dietary styles, allergies, and disliked ingredients can be updated at any time from the profile page.

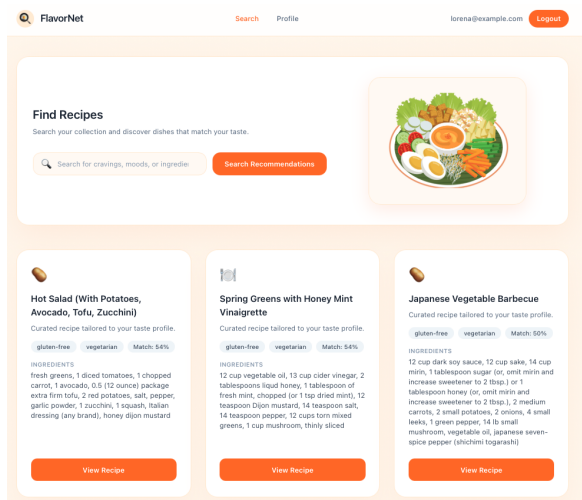


Figure 7: **Inspiration Feed.** Users can browse recipes across cuisines and categories that match their personalized preferences, allergies, and dislike settings.

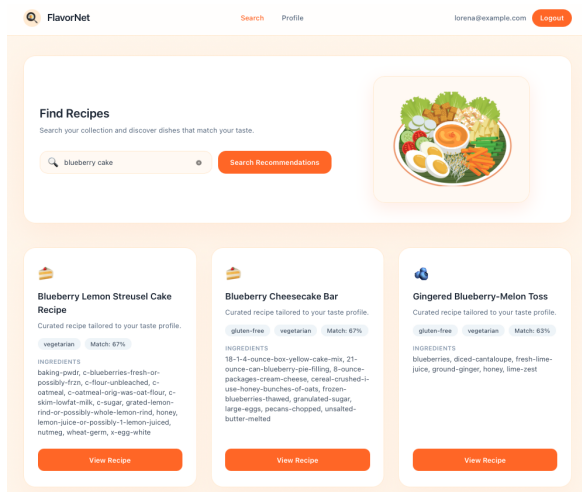


Figure 8: **Semantic Search Results.** Users can enter natural-language queries such as “blueberry cake” or “spicy tofu” and receive personalized, allergy-safe recipe recommendations.